

ЛАБОРАТОРНА РОБОТА 4

РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

4.1 Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

4.2 Результати Code Review одnogрупника: таблиця

№	Рядок коду	Проблема	Категорія	Рекомендація
1	<code>MAX_SKILLS = 5</code>	Число 5 не пояснює своєї природи — незрозуміло чому саме таке обмеження	Magic Number	Винести на рівень модуля як <code>MAX_SKILLS_PER_MEMBER = 5</code> з поясненням у docstring класу
2	<code>self._pending_requests = []</code>	Назва не вказує що зберігаються <code>user_id</code> ; list має $O(n)$ для перевірки належності	Poor Naming + Performance	Замінити на <code>self._pending_request_ids = set()</code> для $O(1)$ lookup; назва відображає вміст
3	<code>if not self.check_in or not self.check_in.active</code>	Ідентична умова повторюється в <code>get_nearby_members</code> і <code>send_connection_request</code>	Code Duplication	Винести в приватний метод <code>_is_active_session()</code> — усунення дублювання за принципом DRY
4	<code>if (not self.check_in or not self.check_in.active or not target.check_in...)</code>	Складна багаторядкова умова виконує дві незалежні перевірки: активність сесій та збіг локацій	Long Condition / Порушення SRP	Extract Method → <code>_at_same_location(target): bool</code> ; кожна перевірка в окремому методі з ясною назвою
5	<code>def update_skills(self, skills)</code>	Метод одночасно валідує дані і зберігає їх — дві відповідальності в одному методі	Порушення SRP	Extract Method → <code>_validate_skills(skills)</code> : повертає очищений список або кидає виняток; <code>update_skills</code> лише зберігає

4.3 Посилання на PR з коментарями Code Review (URL):

<https://github.com/johanmoses79/lr4opi/pull/1>

4.4 Результати рефакторингу власного коду

Зміна 1 — Replace Magic Number with Constant (CR-01)

БУЛО:

```
MAX_SKILLS = 5
```

СТАЛО:

```
# на рівні модуля, поза класом
```

```
MAX_SKILLS_PER_MEMBER = 5
```

```
class Member:
```

```
...
```

```
# у методі:
```

```
if len(skills) > MAX_SKILLS_PER_MEMBER:
```

```
    raise ValueError(f'Max {MAX_SKILLS_PER_MEMBER} skills allowed')
```

ЧОМУ:

Число 5 всередині класу не пояснює свого призначення читачу коду. Перенесення на рівень модуля з описовою назвою MAX_SKILLS_PER_MEMBER робить обмеження явним та легко змінюваним в одному місці без пошуку по всьому класу.

Зміна 2 — Replace Collection: list → set (CR-02)

БУЛО:

```
self._pending_requests = [] # user_id надісланих запитів
```

```
...
```

```
if target.user_id in self._pending_requests: # O(n)
```

```
self._pending_requests.append(target.user_id)
```

СТАЛО:

```
self._pending_request_ids = set()
```

```
...
```

```
if target.user_id in self._pending_request_ids: # O(1)
```

```
self._pending_request_ids.add(target.user_id)
```

ЧОМУ:

Перевірка належності ('in') на списку має лінійну складність O(n) — зі зростанням кількості запитів час пошуку збільшується. Множина (set) використовує хешування і забезпечує O(1). Нова назва _pending_request_ids явно вказує що зберігаються ідентифікатори, а не об'єкти.

Зміна 3 — Extract Method: `_is_active_session()` (CR-03)

БУЛО (в `get_nearby_members`):

```
if not self.check_in or not self.check_in.active:
    raise RuntimeError('You must be checked in...')
```

БУЛО (в `send_connection_request`):

```
if (
    not self.check_in or not self.check_in.active
    or not target.check_in or not target.check_in.active
    ...
):
```

СТАЛО:

```
def _is_active_session(self):
    return self.check_in is not None and self.check_in.active

# у get_nearby_members:
if not self._is_active_session():
    raise RuntimeError('You must be checked in...')

# у send_connection_request:
if not self._is_active_session() or not target._is_active_session():
    raise RuntimeError('Both members must be at the same active location')
```

ЧОМУ:

Ідентична умова перевірки активності сесії дублювалась у двох методах — порушення принципу DRY (Don't Repeat Yourself). Якщо логіка перевірки зміниться (наприклад, додається перевірка часу), довелось би правити в кількох місцях. Extract Method усуває дублювання: єдине місце визначення логіки, покращена читабельність методів.

4.5 Звіт регресійного тестування: скріншот результату pytest / JUnit / Jest — усі тести green після рефакторингу.

Ruff до:

```
PS D:\Downloads> py -m ruff check 'sehsnmock (2).py'
E402 Module level import not at top of file
--> sehsnmock (2).py:133:1

131 | # Патерн: AAA (Arrange / Act / Assert)
132 |
133 | import pytest
    | ~~~~~
134 | from member import Member, CheckIn

E402 Module level import not at top of file
--> sehsnmock (2).py:134:1

133 | import pytest
134 | from member import Member, CheckIn
    | ~~~~~

F811 Redefinition of unused `Member` from line 17
--> sehsnmock (2).py:134:20

133 | import pytest
134 | from member import Member, CheckIn
    | ~~~~~ `Member` redefined here

::: sehsnmock (2).py:17:7

17 | class Member:
    | ~~~~~ previous definition of `Member` here
18 | """
19 | Активний учасник платформи Seshn Networks.

help: Remove definition: `Member`

F811 Redefinition of unused `CheckIn` from line 4
--> sehsnmock (2).py:134:28

133 | import pytest
134 | from member import Member, CheckIn
    | ~~~~~ `CheckIn` redefined here

::: sehsnmock (2).py:4:7

4 | class CheckIn:
  | ~~~~~ previous definition of `CheckIn` here
5 | """Представляє одну сесію присутності учасника в локації."""

help: Remove definition: `CheckIn`
```

Ruff після:

```
PS C:\Users\hiorai> cd D:/Downloads
PS D:\Downloads> py -m ruff check sehsnmockremade.py
All checks passed!
```

Sonarlint до:



The screenshot shows the VS Code editor with a Python file named `sehsnmock (2).py`. The code defines a `CheckIn` class with an `__init__` method. The SonarLint interface is visible at the bottom, showing 10 problems and 6 SonarQube issues. The 'SONARQUBE FINDINGS (ALL FINDINGS)' panel lists several issues related to commented-out code and duplicated literals. The 'SONARQUBE FOR IDE LABS' panel displays a 'Be a Sonar Insider' banner with the SonarQube logo.

```
1 from datetime import datetime
2
3
4 class CheckIn:
5     """Представляє одну сесію присутності учасника в локації."""
6
7     def __init__(self, location_id):
8         self.location_id = location_id
9         self.timestamp = datetime.now()
10        self.active = True
11
```

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS SONARQUBE 6

✓ SONARQUBE FINDINGS (ALL FINDINGS)

✓ sehsnmock (2).py 9+

- Remove this commented out code. (python:S125) [Ln 137, Col 0]
- Define a constant instead of duplicating this literal "ivan@test.com" 23 times. [+2...
- Remove this commented out code. (python:S125) [Ln 189, Col 0]
- Remove this commented out code. (python:S125) [Ln 237, Col 0]
- Define a constant instead of duplicating this literal "olena@test.com" 8 times. [+...
- Remove this commented out code. (python:S125) [Ln 315, Col 0]

SONARQUBE FOR IDE LABS

Be a Sonar Insider

 SonarQube
ide labs

Sonarlint після:



The screenshot shows the VS Code editor with a Python file named `sehsnmockremade.py`. The code defines a `Member` class with `send_connection_request` and `update_skills` methods. The SonarLint interface is visible at the bottom, showing 0 problems and 0 SonarQube issues. The 'SONARQUBE FINDINGS' panel displays 'No SonarQube issues to display.' The 'SONARQUBE FOR IDE LABS' panel displays a 'Be a Sonar Insider' banner with the SonarQube logo.

```
20 class Member:
21     def send_connection_request(self, target):
22
23         if target.user_id in self.pending_request_ids:
24             return f"Request to {target.name} already pending"
25
26         self.pending_request_ids.add(target.user_id)
27         return f"Request sent to {target.name}"
28
29     def update_skills(self, skills):
30         """Валідую та замінює список навичок учасника."""
31         self.pending_request_ids.add(target.user_id)
32
```

PROBLEMS 0 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS SONARQUBE 0

✓ SONARQUBE FINDINGS

No SonarQube issues to display.

SONARQUBE FOR IDE LABS

Be a Sonar Insider

 SonarQube
ide labs

Get early access to our newest features and help shape the future of SonarQube for IDE. Sign up for our Labs program to test new functionality and provide feedback directly to our team.

Radon до:

```
PS D:\Downloads> py -m radon cc 'sehsnmock (2).py' -s
sehsnmock (2).py
  M 59:4 Member.get_nearby_members - B (10)
  M 86:4 Member.send_connection_request - B (8)
  C 17:0 Member - B (7)
  M 40:4 Member.check_in_location - A (5)
  F 224:0 test_checkin_different_location_closes_old_session - A (4)
  M 26:4 Member.__init__ - A (4)
  M 112:4 Member.update_skills - A (4)
```

Radon після:

```
PS D:\Downloads> py -m radon cc sehsnmockremade.py -s
sehsnmockremade.py
  M 65:4 Member.get_nearby_members - B (8)
  M 87:4 Member.send_connection_request - B (6)
  C 20:0 Member - A (5)
  M 29:4 Member.__init__ - A (4)
  M 48:4 Member.check_in_location - A (4)
  M 110:4 Member._validate_skills - A (4)
  C 7:0 CheckIn - A (2)
  M 44:4 Member._is_active_session - A (2)
  M 10:4 CheckIn.__init__ - A (1)
  M 15:4 CheckIn.end_session - A (1)
  M 106:4 Member.update_skills - A (1)
PS D:\Downloads>
```

.7. Підсумкова рефлексія: 3–5 речень про найцінніший досвід, отриманий під час Code Review.

Найціннішим досвідом під час Code Review стало вміння дивитися на власний код очима стороннього читача. Виявилось, що код може працювати коректно — всі 24 тести проходили — але при цьому містити приховані проблеми: дубльовану логіку, неочевидні назви та структурні порушення принципу єдиної відповідальності. Особливо корисним було усвідомлення різниці між «код працює» та «код легко читається і підтримується» — це два різні стандарти якості. Практика Extract Method показала, що навіть невелика зміна — виділення трьох рядків в окремий метод `_is_active_session()` — одразу усуває дублювання в двох місцях і робить решту методів значно зрозумілішою. Code Review як процес формує звичку писати код не лише для машини, а й для наступного розробника, який буде з ним працювати.